Pipeline Raman–IA Fractale : Formules, Algorithmes et Diagrammes (version une colonne)

Tristan Tardif-Morency

2025

1 Introduction

L'analyse Raman exige un prétraitement rigoureux pour transformer les acquisitions instrumentales brutes en données numériques exploitables par des algorithmes d'analyse et d'apprentissage automatique. Les objectifs principaux du prétraitement sont : éliminer les artefacts impulsionnels, corriger la ligne de base instrumentale, aligner et calibrer l'axe spectral, normaliser l'échelle d'intensité, produire une grille de fréquences régulière et extraire des dérivées stables pour accentuer les signatures spectrales.

Ce rapport traduit — pas à pas et mathématiquement — l'implémentation réelle fournie dans le dépôt : chaque module logiciel est décrit, justifié et muni de recommandations de paramètres et de comportement numérique. L'approche est pragmatique : indiquer quand et pourquoi choisir une méthode, donner des bornes de paramètres, signaler les risques numériques (conditionnement, aliasing, perte d'information).

2 Vue d'ensemble du pipeline

Le pipeline s'exécute selon l'ordre immuable suivant :

NoiseFilter → BaselineCorrector → Calibrator → Normalizer → Resampler → Derivator → QCReporter.

Chaque module reçoit et met à jour un objet de contexte `context` contenant au minimum :

$$\texttt{context} = \{\, x,\ y,\ \texttt{meta},\ \texttt{baseline},\ \texttt{qc} \,\},$$

où x est l'axe spectral (units cm^{-1} ou nm), y l'intensité, `meta` contient métadonnées (laser, intégration), `baseline` stocke la baseline estimée, `qc` accumule les indicateurs de qualité.

Remarques d'orchestration :

- Les opérations doivent être numériquement stables et idempotentes : réappliquer un même module sur un spectre déjà traité ne doit pas introduire d'artéfacts majeurs.
- Les modules utilisent des implémentations vectorisées (NumPy/SciPy) et, lorsque possible, des solveurs creux pour la régularisation.
- Les paramètres par défaut doivent privilégier robustesse > sensibilité (éviter la sur-correction).

3 NoiseFilter — détection et réduction du bruit

But : éliminer *cosmic spikes*, atténuer le bruit haute fréquence, laisser intacte la forme des pics.

3.1 Pipeline interne

Opérations faites, dans l'ordre recommandé :

1. détection et remplacement des pics impulsionnels (méthode médiane locale) ;
2. filtrage Savitzky–Golay (SG) pour lissage et (optionnellement) dérivée ;
3. denoising par ondelettes (optionnel) pour structure multi-échelle.

3.2 Détection des spikes (algorithme précis)

Soit $y \in \mathbb{R}^N$. On calcule une estimation lissée :

$$\tilde{y} = \mathrm{SG}(y; w_{\mathrm{sg}}, p).$$

Résidu : $r = y - \tilde{y}$. Estimation robuste de l'écart type :

$$\hat{\sigma} = \frac{\mathrm{median}(|r|)}{0.6745}.$$

Seuil :

$$T = \kappa \cdot \hat{\sigma}, \qquad \kappa \in [3, 8] \text{ (recommandé : 5).}$$

Détection : indices $I = \{i : |r_i| > T\}$. Remplacement :

$$y_i \leftarrow \mathrm{median}\left\{y_{i-w_s}, \ldots, y_{i+w_s}\right\}, \quad w_s \in \{1, 2, 3\}.$$

Pourquoi médiane ? : robustesse face aux impulsions ; la moyenne lisse étale l'impulsion au lieu de la supprimer.

3.3 Savitzky–Golay (implémentation et paramètres)

SG = convolution avec noyau c_k qui minimise l'erreur d'approximation polynomiale d'ordre p sur fenêtre de taille $2K + 1$. Utilisation typique :

$$w_{\mathrm{sg}} = 5 \text{ ou } 7, \quad p \in \{2, 3\}.$$

Choix pratique :
- signaux fins (pics étroits) : petite fenêtre, p élevé (ex. $w = 5, p = 3$) ;
- signaux larges : plus grand w (ex. $w = 11, p = 2$).

La dérivée d'ordre d est fournie par les coefficients dérivés $c^{(d)}$ (disponible via `savgol_filter`).

3.4 Ondelettes — Denoising

Algorithme :

1. Calcul des coefficients DWT jusqu'au niveau J.
2. Estimation $\hat{\sigma}$ sur coefficients de détail les plus fins.
3. Seuillage soft avec $\lambda = \hat{\sigma}\sqrt{2 \log N}$.
4. Reconstruction IDWT.

Utiliser si le bruit est non-gaussien et multi-échelle ; sinon SG suffit et coûte moins cher.

3.5 Paramètres recommandés et complexité

- spike_sigma $\kappa \in [4, 6]$
- spike_window $w_s = 1$ ou 2
- sg_window odd = 5–11, polyorder = 2–3
- wavelet : ''db4'' ou ''sym5'' ; level $J \approx \lfloor \log_2 N \rfloor - 2$

Complexité dominante : DWT $O(N)$, SG $O(N)$; remplacement médian $O(m \cdot N)$ si naïf (utiliser fenêtrage glissant).

4 Baseline correction — AsLS, AirPLS, SNIP

But : estimer z tel que $y - z$ contienne les pics Raman.

4.1 Formulation AsLS (dérivation)

On minimise :

$$J(z) = \sum_{i=1}^{N} w_i (y_i - z_i)^2 + \lambda \sum_{i=1}^{N-2} (z_i - 2z_{i+1} + z_{i+2})^2.$$

Noter $D \in \mathbb{R}^{(N-2)\times N}$ la matrice de seconde différence. En écriture matricielle :

$$J(z) = (y - z)^\top W (y - z) + \lambda \|Dz\|_2^2.$$

Condition d'optimalité :

$$(W + \lambda D^\top D) z = W y.$$

Procédure itérative AsLS :

1. initialiser $w_i = 1$.
2. résoudre le système ci-dessus pour z.
3. mettre à jour $w_i \leftarrow p$ si $y_i > z_i$ et $1 - p$ sinon (ou via une règle continue).
4. répéter jusqu'à convergence (critère : $\|z^{(k+1)} - z^{(k)}\|_\infty < \varepsilon$).

4.2 AirPLS (pratique)

AirPLS modifie w de façon continue et décroissante avec le carré du résidu négatif, favorisant une baseline sous le signal. Converge typiquement en ≤ 30 itérations pour signaux Raman classiques.

4.3 SNIP (courts commentaires)

SNIP est adapté si le fond est très lent et si l'on veut éviter résoudre un système linéaire ; il est cependant plus agressif et peut supprimer des basses fréquences utiles si mal paramétré.

4.4 Implémentation numérique et stabilité

— résoudre le système avec solveur creux (sparse Cholesky / spsolve) ; éviter inversion explicite.
— paramètre λ contrôle la lissage : $\lambda \in [10^2, 10^9]$. Valeurs pratiques : $\lambda \approx 10^6$ pour données bruyantes d'instruments.
— choix p (asymétrie) : $p \in [0.001, 0.05]$ pour sous-estimer la baseline.

4.5 Pseudo-code

```
w = ones(N)
for k in range(max_iter):
    A = W + lambda * D.T @ D    # sparse
    z = spsolve(A, W @ y)
    r = y - z
    update w based on r (AsLS or AirPLS rule)
    if converge: break
```

5 Calibration spectrale et réponse instrumentale

Objectifs : aligner l'axe spectral et compenser la variation spectrale en intensité.

5.1 Détection et appariement des pics

Algorithme de détection (code) :
— détection de maxima locaux sur signal filtré (SG) ;
— seuil minimal d'amplitude (SNR_min) et de largeur (pour ignorer bruit) ;
— extraction de positions $\tilde{\nu}_j$ et amplitudes A_j.

Appariement : résoudre l'association minimale entre $\{\tilde{\nu}_j\}$ et références $\{\nu_k^*\}$ par coût $|\tilde{\nu}_j - \nu_k^*|$ et contrainte de proximité.

5.2 Ajustement polynômial

Minimisation :

$$\min_a \sum_j \left(\nu_j^* - P_r(\tilde{\nu}_j; a)\right)^2,$$

où $P_r(x; a) = \sum_{m=0}^{r} a_m x^m$. Résolution via QR ou `np.polyfit` (numériquement stable pour $r \leq 3$).

5.3 Correction de réponse instrumentale

Si $R(x)$ est fourni (mesure d'étalon), on effectue :

$$y_{\text{corr}}(x) = \frac{y(x)}{R(x) + \varepsilon}, \qquad \varepsilon \approx 10^{-12}$$

Interpolation : utiliser `interp1d` en mode 'linear' ou 'slinear' pour éviter oscillations.

5.4 Remarques pratiques

— Vérifier la monotonie de la transformation P_r (pas d'inversions d'axe).
— Si trop peu de pics de référence : utiliser calibration par ruban ou lampe étalon, ou faire un recalage relatif (cross-correlation).
— Indiquer dans `meta` la transformation appliquée (coefficients du polynôme).

6 Normalisation

But : rendre les spectres comparables en amplitude.

6.1 Méthodes et formules

— **SNV (Standard Normal Variate)** :

$$y' = \frac{y - \bar{y}}{\sigma_y}.$$

Avantage : corrige offset et échelle localement ; sensible aux outliers.

— **Min–Max** :

$$y' = \frac{y - y_{\min}}{y_{\max} - y_{\min}}.$$

Simple mais dépend fortement des extrêmes (seulement si spectres couverts de la même gamme).

— **Area (normalisation par aire)** :

$$y' = \frac{y}{\int y \, \mathrm{d}x}.$$

Préserve la forme relative ; recommandée si l'aire totale est physique (même concentration).

6.2 Recommandations

Choisir SNV pour données empiriques multi-instruments, Area si la quantité intégrée est physiquement signicative, Min–Max uniquement pour visualisation.

7 Rééchantillonnage (Resampler)

But : produire un axe régulier indispensable pour CNN 1D, FFT et comparaisons.

7.1 Grille et interpolation

Générer :

$$x_{\text{new}} = [x_{\min}, x_{\min} + \Delta\nu, \ldots, x_{\max}],$$

avec $\Delta\nu$ choisi selon résolution instrumentale effective (1 cm^{-1} ou 0.5 cm^{-1} courant). Interpolations :

- `linear` : stable, pas d'oscillation ;
- `cubic` / spline : meilleur lissage, mais risque d'overshoot aux bords ;
- recommandation : linear ou cubic with 'extrapolate=False'.

7.2 Anti-aliasing et préfiltrage

Avant décimation (si $\Delta\nu$ augmente), appliquer filtre passe-bas pour éviter aliasing ; dans la plupart des cas, on interpole sans décimer (grid finer or equal).

7.3 Conservation d'aire

Si l'intégrale est physiquement significative, corriger l'interpolation pour conserver l'aire totale (normalisation post-interp).

8 Dérivation et dérivée fractale

But : faire ressortir composantes spectrales overlapping et améliorer séparation de pics.

8.1 Savitzky–Golay dérivée

SG fournit $\frac{d^d y}{dx^d}$ via convolution. Coefficients obtenus analytiquement ; robustes au bruit si la fenêtre est correctement choisie.

8.2 Dérivées fractionnaires (approche numérique)

On peut approximer la dérivée d'ordre réel $D_f \in \mathbb{R}^+$ par une discrétisation Grünwald–Letnikov :

$$\frac{d^{D_f}}{dx^{D_f}} y(x) \approx \frac{1}{\Delta x^{D_f}} \sum_{k=0}^{M} \omega_k^{(D_f)} y(x - k\Delta x),$$

avec poids

$$\omega_0^{(D_f)} = 1, \quad \omega_k^{(D_f)} = \left(1 - \frac{D_f + 1}{k}\right) \omega_{k-1}^{(D_f)}.$$

Prudence : ces poids décroissent lentement ; la somme longue augmente le coût et la sensibilité au bruit. Utiliser régularisation (fenêtrage exponentiel des poids) et adapter M selon SNR.

8.3 Stratégie adaptative

Règle pratique : si SNR > 20 appliquer $D_f \in [1.0, 1.5]$; sinon rester à $d = 1$ ou 2 derivées SG.

9 Contrôle qualité et rapport

Le module calcule des métriques et produit les artefacts suivants : tableau des indicateurs, figures (brut / baseline / corrigé / dérivée), et export Excel.

9.1 Indicateurs calculés

— **SNR** : $20 \log_{10} \frac{\max(y)-\min(y)}{\hat{\sigma}_{\text{noise}}}$.
— **RMSE baseline** : $\sqrt{\frac{1}{N}\sum(y_{\text{raw}} - y_{\text{corr}})^2}$.
— **Peak match score** : fraction de pics de référence retrouvés $\in [0, 1]$.
— **Outliers %** : pourcentage $|z| > 3$.
— **Area change %** : relative difference d'aire.
— **Score composite** :

$$Q = \sum_i \alpha_i S_i, \quad \sum \alpha_i = 1,$$

où S_i sont normalisés (SNR normalized, RMSE negated, match score).

9.2 Seuils pratiques

Proposition par défaut :
— SNR_{min} acceptable : 10 dB ;
— RMSE acceptable : dépend du signal ; utiliser test relatif au bruit estimé ;
— Peak match threshold : ≥ 0.8.

9.3 Format du rapport

PDF contenant :
— entête metadata (fichier, date, paramètres) ;
— figures : superposition brut/baseline/corrigé ; carte de densité des résidus ; peaks markers ;
— table Excel : métriques par spectre + paramètres.

10 Compression spectrale — ajustement de pics et ondelettes

Deux stratégies complémentaires.

10.1 Ajustement paramétrique de pics (Gauss, Lorentz)

Fitter non-linéaire :

$$\hat{y}(x) = \sum_{k=1}^{K} A_k \cdot \exp\Big(- \frac{(x-\mu_k)^2}{2\sigma_k^2} \Big)$$

ou Lorentzienne, estimer $\theta = [A_k, \mu_k, \sigma_k]_{k=1..K}$ par `least-squares` (Levenberg–Marquardt). Initialisation : positions par détection de maxima, amplitudes à maxima locaux, widths par mesure FWHM estimée.

Avantage : compression élevée, paramètres physiquement interprétables. Risque : sur-ajustement si K mal choisi.

10.2 Compression par ondelettes

Conserver M coefficients DWT les plus énergétiques puis reconstruire. Méthode sans modèle — adaptée si la structure spectrale est multi-échelle non paramétrique.

11 Annexe : matrices et aspects numériques

11.1 Matrice D de seconde différence

Définition (taille $(N-2) \times N$) :

$$D = \begin{bmatrix} 1 & -2 & 1 & 0 & \dots & 0 \\ 0 & 1 & -2 & 1 & \dots & 0 \\ \vdots & & & & \ddots & \vdots \\ 0 & \dots & 0 & 1 & -2 & 1 \end{bmatrix}$$

Conditionnement : $D^\top D$ a des valeurs propres croissantes $\propto k^4$; choisir λ pour équilibrer la pénalisation.

11.2 Complexité et performance

- SG, DWT, interpolation : $O(N)$ ou $O(N \log N)$ selon l'implémentation.
- Résolution ALS : résolution d'un système tridiagonal-like (sparse) — quasiment $O(N)$ si exploit sparse Cholesky.
- Recommandation : vectoriser, éviter boucles Python sur chaque point ; utiliser numba si besoin.

12 Paramètres par défaut et bonnes pratiques

- NoiseFilter : `spike_sigma`=5, `sg_window`=7, `sg_polyorder`=3.
- Baseline : method=‘airpls‘ ou ‘als‘, $\lambda = 10^6$, $p = 0.01$, niter=20.
- Calibrator : poly order=2 ou 3, minimal 3 peaks for robust fit.
- Resampler : $\Delta\nu = 1.0$ cm^{-1} par défaut, interpolation ‘linear‘.
- Derivator : SG deriv order 1 with window=9 for moderate noise ; fractional only for SNR$\geq$ 20.

13 Tests, validation et métriques unitaires

- Tests unitaires : round-trip interpolation (resample then inverse compare), conservation d'aire après correction réponse (si exigée).
- Benchmarks : mesurer temps par spectre pour N=2048 ; viser $<$ 10 ms par étape critique (SG/Resample) en vectorisé.
- Validation expérimentale : vérifier que positions de pics de référence sont stables ($<$ 0.5 cm^{-1}).

14 Conclusion

Chaque section ci-dessus fournit : (i) la formulation mathématique exacte, (ii) les algorithmes pratiques, (iii) recommandations de paramètres, (iv) risques numériques et solutions. Le pipeline ainsi formalisé permet une reproduction fidèle des traitements réalisés par le code Python et constitue une base solide pour extension (IA en aval, optimisation adaptative des paramètres, tests instrumentaux).